Docker

Khoa A. Vu & Kelly Pham

Student ID: 030063200 & 03039600

California State University, Long Beach

CECS 327 - Section 01

Dr. Hailu Xu

September 20, 2025

Objective

The purpose of this project was to gain hands-on experience with Docker by:

1. Understanding containerization concepts.

2. Running basic containers (Alpine, Hello-World).

3. Creating a custom Docker image for a Python app.

4. Deploying Nginx with a custom page.

5. Building a multi-container client-server system using Docker Compose.

Implementation

- Nginx Task
    - Created an `index.html` file with the message *"Hello from my custom Nginx!"*.
    - Ran the Nginx container with port mapping and volume mounting.
    - Verified by visiting: http://localhost:8080.
- Multi-Container Setup
    - Server (server.py):
        - A Python TCP server listening on port 5000. Handles client connections, echoes back "Server received your TCP message!".
    - Clients (client.py):
        - Modified to automatically connect to the server, send "Hello from Docker client!", and print the response.
    - Dockerfiles:
        - Each folder (server, client) contains a minimal Dockerfile based on `python:3.9-slim`.

- Docker-compose.yml:
  - Defines one server and two clients, all connected via a custom bridge network `appnet`.

Screenshots

```python
client.py
client > client.py > ...
import socket

def client_program():
    host = 'server' # must match service name in docker-compose.yml
    port = 5000

    client_socket = socket.socket()
    client_socket.connect((host, port))  # connecting

    '''
    message = input("> Type your message (or 'exit' to quit): ")

    while message.lower().strip() != 'exit':
        client_socket.send(message.encode())  # send message that is encoded
        data = client_socket.recv(1024).decode()  # receive response and decode

        print('Received from server: ' + data)

        message = input(" -> ")  # takes input again
    '''
    # Send one test message automatically
    message = "Hello from Docker client!"
    client_socket.send(message.encode())
    data = client_socket.recv(1024).decode()
    print('Received from server: ' + data)

    client_socket.close()  # close connection


if __name__ == '__main__':
    client_program()
```

client.py - connects to the server where it send a request and a response back

server.py

server > server.py > ...

```python
import socket
import threading

'''
DigitalOcean – "Python Socket Programming: Server and Client Example Guide" Kumar, Pankaj; Walia, Anish Singh. "Python Socket Programming: Server and Client Example Guide." DigitalO
GeeksforGeeks – "How to Capture UDP Packets in Python" GeeksforGeeks. "How to Capture UDP Packets in Python." Last updated July 23, 2025. Available at: https://www.geeksforgeeks.org
'''

# Configuration
HOST = '0.0.0.0'  # listen on all interfaces (important for Docker)
PORT = 5000       # common port for custom python server
USE_TCP = True    # Set to False to use UDP

def handle_tcp_client(conn, addr):
    '''Handles a single TCP client connection in its own thread'''
    print(f"[TCP] Connected by {addr}")
    while True:
        data = conn.recv(1024)  # Receive up to 1024 bytes
        if not data:
            break
        print(f"[TCP] Received from {addr}: {data.decode()}")
        conn.sendall(b"Server received your TCP message!") # Echo back a response
    print(f"TCP client {addr} disconnected.")
    conn.close()

def start_tcp_server():
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.bind((HOST, PORT))
    sock.listen()
    print(f"[TCP] Server listening on port {PORT}")
    while True:
        conn, addr = sock.accept() #accept connnection
        print(f'Connected by {addr}')
        thread = threading.Thread(target=handle_tcp_client, args=(conn, addr)) #handles multiple clients
        thread.start()


def start_udp_server():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((HOST, PORT)) #binding socket

    print(f"[UDP] Server listening on port {PORT}")

    while True:
        # receiving data from the socket
        data, addr = sock.recvfrom(1024) # will read a max of chosen 1024 bytes from message
        print(f"[UDP] Received from {addr}: {data.decode()}") # decode the bytes and print sender's info
        sock.sendto(b"Server received your UDP message!", addr)
```

```python
'''def start_udp_server():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.bind((HOST, PORT))
        print(f"[UDP] Server listening on port {PORT}")
        while True:
            data, addr = s.recvfrom(1024) # UDP is connectionless
            print(f"[UDP] Received from {addr}: {data.decode()}")
            s.sendto(b"Message received", addr)
        conn.close()'''

if __name__ == "__main__":
    if USE_TCP:
        start_tcp_server()
    else:
        start_udp_server()

'''
# creating a UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, UDP_PORT)) #binding socket

print(f"Listening for UDP packets on {UDP_IP}:{UDP_PORT}")

while True:
    # receiving data from the socket
    data, addr = sock.recvfrom(1024) #will read a max of chosen 1024 bytes from message
    print(f"Received packet from {addr}: {data.decode('utf-8')}") #decode the bytes and print sender's info
'''
```

server.py - listens for client connects, processes requests, sends request back to client

```
Dockerfile
client > Dockerfile > FROM
FROM python:3.9-slim
WORKDIR /app
COPY client.py .
CMD ["python", "client.py"]
```

Dockerfile

```
docker-compose.yml
docker-compose.yml
version: '3.8'

Run All Services
services:
  Run Service
  server:
    build: ./server
    container_name: dual_protocol_server
    ports:
      - "5000:5000/tcp"
      - "5000:5000/udp"
    networks:
      - appnet

  Run Service
  client1:
    build: ./client
    container_name: client1
    depends_on:
      - server
    networks:
      - appnet

  Run Service
  client2:
    build: ./client
    container_name: client2
    depends_on:
      - server
    networks:
      - appnet

networks:
  appnet:
    driver: bridge
```

docker-compose.yml - configures server and client services

Challenges & Solutions

- Problem: Clients using `input()` caused crashes (`EOFError`) in Docker Compose.
    - Solution: Modified clients to send a test message automatically.
- Problem: Port 8080 was already in use during Nginx setup.
    - Solution: Stopped the old container or used a different host port.

Contributions

- Kelly Pham: Implemented client code, server code, Dockerfile, and docker-compose.yml.
- Khoa Vu: Debugged and tested the server and client code, Dockerfile, and Docker Compose setup, while also creating the README, project report, and video recording.

- Both: Review Nginx task and documentation.

Conclusion

Overall, this project provided practical experience with Docker, especially showing how to build images using Dockerfiles, run and test containers, deploy a web server with custom content, and last but not least, create a working distributed system (server + clients) via Docker Compose.

Result

video of code execution & output

References

1) DigitalOcean – "Python Socket Programming: Server and Client Example Guide" Kumar, Pankaj; Walia, Anish Singh. "Python Socket Programming: Server and Client Example Guide." DigitalOcean, updated February 21, 2025. Available at: https://www.digitalocean.com/community/tutorials/python-socket-programming-server-client
2) GeeksforGeeks – "How to Capture UDP Packets in Python" GeeksforGeeks. "How to Capture UDP Packets in Python." Last updated July 23, 2025. Available at: https://www.geeksforgeeks.org/python/how-to-capture-udp-packets-in-python/#